SMART STORAGE TANK LEVEL & USAGE PREDICTION SYSTEM

For Petrochemical Plants

Milestone 4 — Analytics Output & Dashboard Visualization

TEAM-7
MITHUN CHANDERAN
AARON ROHIT
RAHUL R
ABINAND

Christ University Internship
Industry Partner: L&T Construction

Sensitivity: LNT Construction Internal Use

1. Milestone Overview

Milestone 4 transforms the machine learning predictions produced in Milestone 3 into a structured analytical output layer and a fully interactive Power BI Desktop dashboard. This milestone closes the loop between the IoT hardware pipeline and operational decision-making by delivering real-time tank status, consumption trends, risk alerts, and prediction accuracy metrics in a single consolidated view accessible to plant operators.

The milestone is delivered in two parts: a Google Colab export pipeline that generates three structured CSV files from the Milestone 3 prediction DataFrame, and a Power BI Desktop dashboard built on those files with live data refresh achieved through a Python-based auto-refresh solution that fetches ThingSpeak data every 15 seconds and automatically triggers a dashboard refresh.

1.1 Milestone Objectives

- Enrich the Milestone 3 prediction DataFrame with all KPI columns required by Power BI visualisations.
- Export a master CSV (milestone4_dashboard_data.csv) containing all predictions and derived metrics.
- Export a live latest CSV (milestone4_live_latest.csv) with the most recent reading per tank.
- Export a daily summary CSV (milestone4_daily_summary.csv) for trend analysis.
- Build an 8-visual Power BI Desktop dashboard covering fill levels, risk alerts, consumption, and prediction accuracy.
- Implement a Python-based real-time auto-refresh solution to update the dashboard every 15 seconds.
- Define DAX measures for KPI cards and calculated risk metrics.

2. Structured Data Output (Google Colab)

Three CSV files are generated from the Colab pipeline as Milestone 4 Step cells appended to the Milestone 2 and 3 notebook. Each file serves a distinct role in the Power BI data model.

2.1 Output File Summary

File	Role	Contents
milestone4_dashboard_data.csv	Primary source	All predictions with 35 KPI columns. One row per tank per timestamp.
milestone4_live_latest.csv	Gauge & card visuals	Single latest row per tank. Drives current fill gauges and status cards.
milestone4_daily_summary.csv	Trend analysis	Daily aggregates per tank: avg/min/max fill, total consumption, alert counts.

2.2 Enriched KPI Columns

The following columns are computed and appended to the DataFrame in the Milestone 4 enrichment step before CSV export:

Column	Type	Description
date	string	Date portion of timestamp (YYYY-MM-DD)
hour_label	string	Formatted hour string (e.g. 14:00) for axis grouping

day_of_week	string	Full day name (Monday through Sunday)
week_number	integer	ISO week number for weekly trend grouping
risk_colour_hex	string	Hex code for conditional formatting: #2ECC71 / #F39C12 / #E74C3C
is_critical	integer	Binary flag: 1 if CRITICAL, else 0
is_warning	integer	Binary flag: 1 if WARNING, else 0
is_normal	integer	Binary flag: 1 if NORMAL, else 0
capacity_used_pct	float	Predicted fill as percentage of tank capacity — drives gauge visuals
capacity_free_pct	float	100 − capacity_used_pct
time_to_empty_mins	float	time_to_empty_hours × 60 for easier dashboard display
abs_error_L	float	Absolute prediction error in litres
error_band	string	Accuracy bucket: Excellent (<100L) / Good / Fair / Poor (>1000L)
running_usage_L	float	Cumulative consumption per tank over time

3. Power BI Data Model

Power BI Desktop is used to build the dashboard. Three CSV tables are loaded and related in Model View to form the data model supporting all dashboard visuals.

3.1 Tables Loaded

#	CSV File	Table Name	Primary Use
1	milestone4_dashboard_data.csv	TankPredictions	Line charts, scatter, error histogram, slicers
2	milestone4_daily_summary.csv	DailySummary	Daily consumption column chart, trend analysis
3	milestone4_live_latest.csv	LiveLatest	Gauge visuals, KPI cards, time-to-empty bar chart

3.2 Table Relationships

From Table : Column	To Table : Column	Cardinality	Direction
TankPredictions : tank_id	LiveLatest : tank_id	Many : One	Single
TankPredictions : date	DailySummary : date	Many : One	Single

4. DAX Measures

All DAX measures are created in the TankPredictions table via Modeling → New Measure. These power the KPI cards and risk classification logic across the dashboard.

4.1 Fill Level & Volume Measures

Tank1_Fill_%

```
Tank1_Fill_% =
VAR dist = AVERAGE(ThingSpeakLive[tank_1_level_cm])
RETURN ROUND((400 - dist) / 400 * 100, 1)
```

Tank1_Litres

```
Tank1_Litres =
VAR dist    = AVERAGE(ThingSpeakLive[tank_1_level_cm])
VAR fillH_m = (400 - dist) / 100
RETURN ROUND(3.14159 * 4 * fillH_m * 1000, 0)
```

4.2 Risk Status Measures

Risk status is derived by converting the raw ultrasonic distance reading (cm) to fill percentage, then applying the three-tier threshold logic. The formula below is applied for each tank:

Tank1_Status

```
Tank1_Status =
VAR dist    = AVERAGE(ThingSpeakLive[tank_1_level_cm])
VAR fillPct = (400 - dist) / 400 * 100
RETURN
    IF(fillPct <= 10, "CRITICAL",
    IF(fillPct <= 25, "WARNING",
    "NORMAL"))
```

The same pattern is applied for Tank2_Status and Tank3_Status using their respective distance fields.

4.3 KPI Card Measures

Total Current Volume (L)

```
Total Current Volume =
SUMX(
    FILTER(TankPredictions,
        TankPredictions[timestamp] = MAX(TankPredictions[timestamp])),
    TankPredictions[predicted_fill_litres]
)
```

Overall Fill %

```
Overall Fill % =
DIVIDE([Total Current Volume (L)], 150796.5, 0) * 100
-- 150796.5 = 3 tanks × 50,265.5 L
```

Avg Time To Empty

```
Avg Time To Empty =
AVERAGEX(
    FILTER(TankPredictions,
        TankPredictions[time_to_empty_hours] < 999),
    TankPredictions[time_to_empty_hours]
)
```

Prediction MAE

```
Prediction MAE =
AVERAGEX(TankPredictions,
    ABS(TankPredictions[prediction_error_L]))
```

5. Power BI Dashboard

The completed dashboard is built in Power BI Desktop on a 1920 × 1080 canvas with a dark navy (#1A2B3C) background. Eight visuals are arranged across three rows covering all four focus areas: fill levels and trends, risk alerts, consumption rates, and predicted versus actual comparison.

5.1 Dashboard Visual Inventory

#	Visual Title	Type	Fields & Purpose
1	KPI Cards Row	Card ×4	Total Volume, Overall Fill %, Critical Tank Count, Avg Time to Empty
2	Tank Fill Level Over Time	Line chart	predicted_fill_litres by timestamp per tank. Threshold lines at 5,027 L (CRITICAL) and 12,566 L (WARNING).
3	Risk Alert Distribution	Donut chart	Count of readings by predicted_risk_level. Colour-coded: green / amber / red.
4	Current Fill % per Tank	Gauge ×3	capacity_used_pct from LiveLatest. One gauge per tank. Target line at 25% (WARNING threshold).
5	Predicted vs Actual Fill	Scatter chart	current_fill_litres (X) vs predicted_fill_litres (Y) by tank_id. Trend line confirms R^2 = 1.000.
6	Daily Consumption Trend	Clustered column	total_consumption_L by date from DailySummary, split by tank_id.
7	Time to Empty by Tank	Bar chart	time_to_empty_hours from LiveLatest per tank. Conditional colours: <2h red, <8h amber, ≥8h green.
8	Prediction Error Distribution	Column chart	Count of records by error_band (Excellent / Good / Fair / Poor).

5.2 Risk Classification Thresholds Used in Dashboard

Risk Level	Fill % Threshold	Volume Equivalent	Colour Code
CRITICAL	≤ 10%	≤ 5,027 L	#E74C3C (Red)

WARNING	10.1% – 25%	5,027 – 12,566 L	#F39C12 (Amber)
NORMAL	> 25%	> 12,566 L	#2ECC71 (Green)

5.3 Dashboard Slicers

- Tank ID slicer (dropdown) — filters all visuals to a single selected tank
- Date range slicer (between) — restricts the time window displayed across all charts
- Risk level slicer (checkbox list) — isolates CRITICAL, WARNING, or NORMAL readings

6. Real-Time Auto-Refresh Implementation

Power BI Desktop's built-in Page Refresh feature requires DirectQuery mode, which the ThingSpeak Web connector does not support natively. A Python-based two-script solution is implemented instead, achieving the same 15-second real-time refresh behaviour without any Power BI configuration limitations.

6.1 Architecture

Component	Script	Function
Data Fetcher	fetch_thingspeak.py	Connects to ThingSpeak REST API every 15 seconds. Converts raw ultrasonic distances to volume in litres using cylindrical geometry. Computes fill %, risk level, and time-to-empty. Writes results to live_tanks.csv.
Dashboard Refresher	auto_refresh_powerbi.py	Uses pyautogui and pygetwindow libraries to locate the Power BI Desktop window, bring it to focus, and simulate an F5 keypress every 15 seconds, triggering a data source re-read.

6.2 Data Fetcher — fetch_thingspeak.py

The fetch script connects to the ThingSpeak REST API endpoint, extracts the latest sensor reading, applies the cylindrical volume formula ($V = \pi r^2 h$, r = 2m, h = 4m), and writes a 3-row CSV with one row per active tank:

```
Endpoint : https://api.thingspeak.com/channels/3327870/feeds.json?results=1
Fields   : field1 (Tank 1 distance), field3 (Tank 2), field5 (Tank 3), field7 (Usage)
Output   : live_tanks.csv — columns: tank_id, distance_cm, fill_litres,
           fill_pct, risk_level, time_to_empty_hrs, total_usage_L, last_updated
Interval : 15 seconds (matches ESP32 upload frequency)
```

6.3 Dashboard Refresher — auto_refresh_powerbi.py

The refresh script runs concurrently in a separate terminal. It uses pygetwindow to locate the Power BI Desktop process by window title, activates the window, and sends an F5 keypress via pyautogui every 15 seconds:

```
Library  : pyautogui, pygetwindow
Action   : gw.getWindowsWithTitle('Power BI Desktop') → win.activate() → F5
Interval : 15 seconds
Install  : pip install pyautogui pygetwindow
```

6.4 End-to-End Real-Time Data Flow

Step	Component	Action
1	ESP32-S3	Reads HC-SR04 sensors and uploads distance readings to ThingSpeak every 15 seconds
2	ThingSpeak Cloud	Stores timestamped field entries accessible via REST API
3	fetch_thingspeak.py	Fetches latest entry → converts distance to volume → classifies risk → writes live_tanks.csv
4	auto_refresh_powerbi.py	Detects Power BI Desktop window → sends F5 keypress → Power BI re-reads live_tanks.csv
5	Power BI Dashboard	All visuals update: gauges show current fill %, status cards show NORMAL/WARNING/CRITICAL, line chart extends, cards update

7. Volume Conversion in Power BI

Since raw ThingSpeak data contains ultrasonic distance readings (cm) rather than volumes, the Power BI data model applies the same cylindrical geometry conversion used in the ESP32 firmware and Milestone 3 Python pipeline.

7.1 Conversion Logic

Parameter	Value / Formula
Tank radius	2.0 m
Tank height	4.0 m = 400 cm
Liquid height (cm)	400 − distance_cm
Volume (litres)	$\pi \times 2^2 \times$ (liquid_height_m) × 1000
Fill percentage	(volume / 50,265.5) × 100
Power Query M formula	= (400 - [tank_1_level_cm]) / 400 * 50265.5

8. Saved Artifacts

File	Format	Description
milestone4_dashboard_data.csv	CSV	Master prediction dataset — 35 columns, all KPI fields. Primary Power BI source.
milestone4_live_latest.csv	CSV	Latest reading per tank — drives gauge and card visuals.
milestone4_daily_summary.csv	CSV	Daily aggregates per tank — avg fill, consumption totals, alert event counts.

live_tanks.csv	CSV	Real-time output of fetch_thingspeak.py — refreshed every 15 seconds for live dashboard feed.
fetch_thingspeak.py	Python	ThingSpeak data fetcher with volume conversion and risk classification.
auto_refresh_powerbi.py	Python	Power BI Desktop auto-refresh script using pyautogui and pygetwindow.
SmartTank_M4_Dashboard.pbix	Power BI	Complete Power BI Desktop report file with all 8 visuals, DAX measures, and slicers.

9. Project Deliverables & Milestones

M	Title	Key Deliverables	Status
1	System Architecture Design	Circuit, pin mapping, ThingSpeak firmware	**Complete**
2	Dataset Generation & ML Model Development	Synthetic dataset, RF Regressor & Classifier (100% accuracy)	**Complete**
3	Live Data Simulation & Model Integration	ThingSpeak fetch, volume conversion, live ML predictions, prediction CSV	**Complete**
4	Analytics Output & Dashboard Visualization	3 structured CSVs, Power BI dashboard (8 visuals), Python real-time refresh, DAX measures	**Complete**
5	Result Analysis & Industry Interpretation	Final report, safety improvement demonstration, inventory management insights	Upcoming

10. Conclusion

Milestone 4 delivers the complete analytics and visualisation layer of the Smart Storage Tank Level and Usage Prediction System. Three structured CSV files are exported from the Colab pipeline, carrying 35 columns of operational KPIs derived from the Milestone 3 ML predictions. A Power BI Desktop dashboard is constructed on these files, presenting eight visuals that give plant operators an immediate understanding of tank fill levels, depletion timelines, risk alerts, and ML prediction accuracy.

Real-time data refresh is achieved through a two-script Python solution: fetch_thingspeak.py continuously retrieves the latest ThingSpeak sensor readings, applies cylindrical volume conversion (r = 2m, h = 4m), and writes an updated CSV every 15 seconds, while auto_refresh_powerbi.py automatically triggers a dashboard reload at the same interval. Together, these scripts replicate the behaviour of Power BI's native Page Refresh feature and ensure the dashboard stays synchronised with the live ESP32 sensor stream.

With Milestones 1 through 4 now complete, the project has delivered a full end-to-end pipeline — from physical sensor measurement through IoT cloud upload, machine learning prediction, and live operational dashboard. Milestone 5 will consolidate these results into a final industry interpretation report demonstrating the safety, efficiency, and inventory management benefits of the system.